ACME: Agentic Contract-Based Multilevel Engineering for High-Assurance Complex Systems

PI: Pierluigi Nuzzo, Associate Professor, Department of Electrical Engineering and Computer Sciences, University of California, Berkeley

Cash funding needed: $80,000–$100,000
AWS Promotional Credits needed: $25,000–$40,000

Amazon contact (optional): [Amazon contact name], [email]

Abstract - Large language model (LLM) agents are increasingly employed to design and verify complex systems, such as register-transfer level (RTL) hardware, controllers for teams of robots, and workflows of cooperating software agents. Their design and verification require decomposing system-level requirements into component-level specifications. Decomposing temporal specifications is a hard problem: distributed reactive synthesis is undecidable in general, and finding a decomposition whose components are realizable and whose composition is correct is still largely an art. This project aims to develop the theory and the computational tools for sound agentic decomposition based on assume-guarantee (A/G) contracts. The proposed framework, termed Agentic Contract-Based Multilevel Engineering (ACME), uses agents as heuristics that propose contracts at every level of a design hierarchy, while the soundness of each step is decided by a contract engine together with model checking and reactive synthesis back ends. The design process is itself multi-agent, since an agent at each level proposes the contracts for the level below, and the systems it targets include both single-agent and multi-agent systems. We will address three open problems: the decomposition of GR(1) contracts into realizable component contracts, including sound approximations of the contract quotient within GR(1) and circular reasoning across components; the preservation of temporal obligations when long-horizon agents compact their context or delegate tasks to sub-agents; and the use of counterexamples, unrealizable cores, and certificates as formal feedback for repair and reuse. ACME will be built on CHASE, our open-source contract-based design framework, and evaluated on hardware, multi-robot, and multi-agent software benchmarks.

Keywords – assume-guarantee contracts, compositional verification, reactive synthesis, GR(1), neurosymbolic reasoning, LLM agents, hardware formal verification, autoformalization.

Introduction

Next-generation computing systems, from custom silicon to fleets of autonomous robots and workflows of cooperating AI agents, are increasingly designed and verified with the help of LLM agents. LLMs generate assertions and invariants for hardware [1, 2] and programs [3], while formal tools judge individual properties and the decomposition of the problem rests on the agent. Decomposition, however, is where most of the difficulty lies. End-to-end properties of large systems, such as in-order data delivery across a chain of hardware modules or mutual exclusion over a fleet of robots, often remain undetermined when checked on the flat design. In our experience, agents that decompose such properties without supervision may report timed-out runs as proofs or rely on circular assumptions across lemmas. Long-horizon agents face a related problem: when an agent compacts its context or delegates a subtask, the rules it must follow are summarized in natural language and may be lost [4].

Contract-based design offers a principled foundation for both problems. An A/G contract specifies what a component guarantees under assumptions on its environment [5]. A decomposition is correct when the composition of the component contracts refines the system contract, and the algebra of contracts provides operators, such as the quotient, that compute the weakest specification a missing component must satisfy [6, 7]. Existing contract-based tools, including CHASE [8], Pacti [9], and OCRA [10], assume that contracts are written by human designers, and existing decomposition methods for synthesis rely on a fixed library

of components [11]. Letting agents write the contracts raises several scientific questions. The existence of a decomposition into realizable components is undecidable for general architectures [12], so an agent may search indefinitely unless each failure is explained in a form that excludes whole families of candidates. The contract operators must also remain computable within the fragments that synthesis tools accept. For instance, the guarantee of the quotient of two GR(1) contracts contains the negation of assumptions and in general falls outside the GR(1) fragment.

This project aims to address these questions by building on our prior work. CHASE [8] provides an extensible contract algebra and interfaces to verification and synthesis tools, and has recently been extended with an interface for LLM agents. Our work on contract-based architecture exploration [13, 14] has shown that infeasibility certificates can be generalized via subgraph isomorphism to prune large design spaces, and received the Best Paper Award at DATE 2024. ACIES [15] uses LLM agents to decompose multi-robot requirements into GR(1) contracts that are certified and synthesized into controllers, and our ongoing work applies the same principle to RTL verification. ACME will unify these efforts.

Methods

ACME represents a design as a directed acyclic graph of contracts, where each node is a component with a contract over its ports. At an internal node, an agent proposes one contract for each subcomponent. The proposal is accepted only if the component contracts are consistent, bounded checks on the actual components find no counterexample, and their composition refines the parent contract. Each subcomponent is then discharged by a back end or, if this does not terminate within a budget, further decomposed. When a node cannot satisfy its contract, it returns a witness to its parent, which revises the decomposition while certified subtrees are preserved. The leaves depend on the domain: a hardware module whose contract a model checker proves directly, a task that one LLM agent completes within its context or a piece of deterministic code, or a continuous-time robot controller certified, for instance, by a control barrier function or a Hamilton-Jacobi reachability analysis. The research is organized in three thrusts.

Thrust 1: Decomposition of temporal contracts. We will study the decomposition of a system contract in the GR(1) fragment [16] into realizable component contracts over a given architecture. First, we will identify classes of architectures and contracts for which the existence of a decomposition is decidable, starting from pipeline architectures, for which distributed synthesis is decidable [12], and use the agent as a search heuristic outside these classes. Second, we will develop sound GR(1) approximations of the quotient. An approximation that strengthens the guarantees of the exact quotient can be solved directly by a synthesis tool, and any controller synthesized for it is correct for the composition. When the approximation is unrealizable, its unrealizable core indicates which terms to weaken in the next approximation. Third, we will handle circular dependencies by restricting the guarantees that can be used as assumptions to safety properties, adopting a circular A/G rule with an inductive side condition [17, 18], and characterizing the class of decompositions this rule admits. Fourth, as learned clauses prune the search of a SAT solver, we will generalize the unrealizable core of a failed decomposition, via subgraph isomorphism [13], into a constraint that excludes every candidate assigning an isomorphic component the same terms.

Thrust 2: Temporal obligations in long-horizon agents. Tasks with long horizons, hundreds of tools, and long policy documents require an agent to compact its context and to delegate subtasks, and both steps can lose the rules the agent must follow [4]; whether one agent or several perform better depends on the structure of the task [19]. We will keep these rules in a library of contracts in linear temporal logic on finite traces (LTL_f) [20] stored outside the context window, so that compaction cannot remove them. The library is populated from the instructions and from the context at each compaction, and is extended with contracts mined from execution traces [21], while consistency, vacuity [22], and well-separation [23] checks reject faulty formalizations. Delegation then becomes a decomposition step: a parent agent passes

to each sub-agent a task description, a context, and a set of contracts, either inherited or generated for it, and the engine checks that the composition of the sub-agent contracts refines the parent contract. A central question is how to generate sub-agent contracts that refine the parent by construction, since a sub-agent may need weaker assumptions, stronger guarantees, and conditions on its siblings, such as sending a confirmation only after a payment; the quotient of the parent contract by the contracts of the siblings gives the weakest admissible contract. The contracts also bound the tools exposed to the agent at each step. We will cast the choice between one agent and several as a contract-based architecture exploration problem [13, 14]. Enforcement at run time reduces to monitoring [24] and is outside the scope of this project.

Thrust 3: Formal feedback, repair, and reuse. The contract engine separates a contract algebra independent of the formalism, contract-level checks reduced to validity or satisfiability, and analyses specific to a formalism, such as unrealizable cores for GR(1). Every failed check returns a counterexample trace or a minimal core identifying the terms responsible for the failure. A Model Context Protocol (MCP) server and an agent skill expose all the operations to LLM agents, while no LLM is part of the engine. A property is claimed only after the back-end result has been read and the vacuity checks have passed, and bounded or timed-out runs count as undetermined. Refinement failures are first repaired mechanically using the quotient, and the agent is invoked only when this repair does not succeed. A certified component can be reused in another design after its assumptions are rechecked. Finally, we will measure the coverage of certified contracts by injecting faults into the components.

Applications and preliminary results. We will evaluate ACME in three domains. In hardware verification, component contracts are SystemVerilog assertions (SVAs) with ghost-state models, verified with Cadence Jasper and, in this project, the open-source SymbiYosys. Our prototype certified an end-to-end in-order delivery property of an asynchronous link from the HierSVA benchmark with 25 Jasper runs and about 16,000 LLM input tokens, which flat Jasper did not decide in the same time. The decomposition found at FIFO depth 8 certified depths 16 to 512 with no further LLM calls. An agent with the same tools and no protocol used 24 times as many tokens without completing the proof. In multi-robot control, ACIES [15] decomposes mission requirements into GR(1) contracts and synthesizes controllers; in a three-level decomposition with 42 nodes, the contract library closed 26 of the 39 leaves. For LLM agents, we will start from a single-agent contract-based harness and add context compaction, contract extraction, trace mining, and contract-checked delegation one at a time, on long-horizon tasks from AppWorld [25] and policy-heavy benchmarks, against the best summarization-based compaction as baseline.

Expected results

Months	Research result	Deliverable
1–3	Decomposition protocol; sound GR(1) quotient approximations (Thrust 1)	Technical report; CHASE release with MCP server and agent skill
4–6	Circular A/G rule for safety guarantees with a soundness proof; generalization of unrealizable cores (Thrust 1)	Open-source hardware flow with SymbiYosys on AWS; RTL benchmark with faulty variants
7–9	Contract library from context and traces; contract generation for delegation (Thrust 2); repair and certificate reuse (Thrust 3)	Hardware and multi-robot evaluation; conference paper
10–12	Choice between one agent and several (Thrust 2); contract coverage (Thrust 3)	Staged evaluation on long-horizon agent tasks; tool paper on CHASE; presentation to Amazon teams

Beyond the first year, we expect these results to extend to further decidable classes of architectures and to contracts with probabilistic guarantees. Contracts compiled into reward machines [26, 27] also give stepwise rewards that are checked formally, which we plan to use to fine-tune small open models. All code, benchmarks, and run traces will be released under open-source licenses.

Funds needed

Cash funding ($80,000–$100,000). Partial support of one graduate student researcher working on the theory, the framework, and the benchmarks (stipend and fees, [amount]), and travel to present the results at conferences ([amount]). The upper end of the range would add partial support of a second student working on Thrust 2. No indirect expenses are included.

AWS Promotional Credits ($25,000–$40,000). Amazon Bedrock for agent inference, since our hardware and robotic pipelines already invoke Claude models through Bedrock; Amazon EC2 compute-optimized instances for parallel runs of nuXmv, slugs, and SymbiYosys; and Amazon S3 for benchmarks and run traces. Experiments that require Cadence Jasper will run on campus license servers.

Additional information

1. Application domain and analysis techniques. Compositional verification and design of hardware, multi-robot systems, and LLM agents, using A/G contracts, linear temporal logic (LTL) and LTL_f model checking (nuXmv), satisfiability modulo theories (SMT) reasoning, GR(1) synthesis (slugs), and RTL property verification (Jasper, SymbiYosys).

2. Current applications. CHASE has been used for requirement formalization and validation of cyber-physical systems [8], contract-based architecture exploration [13, 14], and consistency and availability analysis of distributed systems in collaboration with Lockheed Martin [28]. Our RTL flow runs on OpenTitan, HierSVA, and riscv-formal designs; ACIES runs on multi-robot warehouse and charging tasks.

3. Potential applications to Amazon. Verification of AWS silicon, where end-to-end properties span many modules; controllers with guarantees for Amazon Robotics fleets sharing chargers, aisles, and docks; multi-agent workflows on Amazon Bedrock in which the obligations of each agent are preserved across delegation; and consistency and vacuity analysis of formalized policies, complementary to Automated Reasoning checks in Amazon Bedrock Guardrails.

4–5. Prior Amazon Research Awards. The PI has not received an Amazon Research Award, and the project team has not received cash funding or AWS Promotional Credits from Amazon in the past three years.

6. Assumptions and soundness. A certified result is sound when (i) the back ends return correct verdicts; (ii) the translation of contracts into each back end preserves their semantics, which we ensure by restricting SystemVerilog assertions to an LTL-expressible subset; (iii) every composition step is acyclic or satisfies the side condition of the circular rule in Thrust 1; and (iv) the top-level specification is correct, which is why extracted contracts are checked for consistency and vacuity and confirmed by the user. Bounded results are never counted as proofs, and LLM outputs are never part of the trusted base. Quotient approximations are used only in the direction that preserves refinement.

7. Tools. *License:* CHASE and the CHASE MCP server are released under the BSD 3-Clause license, the RTL verification flow under Apache 2.0, and ACIES under the MIT license. *On-boarding material:* a website with a usage guide and an API reference, a Docker image with all the solvers, and the agent skill with worked examples; a tutorial will be added. *Maintenance:* CHASE is actively maintained, with over 90 commits and continuous integration in the last three months. It has one active contributor.

Appendix A – Reference

[1] Orenes-Vera, Marcelo, et al. “AutoSVA: Democratizing Formal Verification of RTL Module Interactions.” DAC. 2021.

[2] Kang, Minwoo, et al. “FVEval: Understanding Language Model Capabilities in Formal Verification of Digital Hardware.” arXiv:2410.23299. 2024.

[3] Wu, Haoze, Clark Barrett, and Nina Narodytska. “Lemur: Integrating Large Language Models in Automated Program Verification.” ICLR. 2024.

[4] Cemri, Mert, et al. “Why Do Multi-Agent LLM Systems Fail?” arXiv:2503.13657. 2025.

[5] Benveniste, Albert, et al. “Contracts for System Design.” Foundations and Trends in Electronic Design Automation. 2018.

[6] Incer, Inigo, Albert Benveniste, and Alberto Sangiovanni-Vincentelli. “Some Algebraic Aspects of Assume-Guarantee Reasoning.” arXiv:2309.08875. 2023.

[7] Incer, Inigo. “The Algebra of Contracts.” Ph.D. thesis, UC Berkeley, EECS-2022-99. 2022.

[8] Nuzzo, Pierluigi, et al. “CHASE: Contract-Based Requirement Engineering for Cyber-Physical System Design.” DATE. 2018.

[9] Incer, Inigo, et al. “Pacti: Scaling Assume-Guarantee Reasoning for System Analysis and Design.” arXiv:2303.17751. 2023.

[10] Cimatti, Alessandro, Michele Dorigatti, and Stefano Tonetta. “OCRA: A Tool for Checking the Refinement of Temporal Contracts.” ASE. 2013.

[11] Iannopollo, Antonio, Stavros Tripakis, and Alberto Sangiovanni-Vincentelli. “Specification Decomposition for Synthesis from Libraries of LTL Assume/Guarantee Contracts.” DATE. 2018.

[12] Pnueli, Amir, and Roni Rosner. “Distributed Reactive Systems Are Hard to Synthesize.” FOCS. 1990.

[13] Xiao, Yifeng, et al. “Efficient Exploration of Cyber-Physical System Architectures Using Contracts and Subgraph Isomorphism.” DATE. 2024.

[14] Xiao, Yifeng, and Pierluigi Nuzzo. “Contract-Based Architecture Exploration of Cyber-Physical Systems via Satisfiability Modulo Convex Programming.” DATE. 2026.

[15] Xiao, Yifeng, et al. “ACIES: A Self-Evolving Agentic Contract-Based Framework for Cyber-Physical System Control Synthesis.” REALM Workshop at EMNLP. 2026.

[16] Bloem, Roderick, et al. “Synthesis of Reactive(1) Designs.” Journal of Computer and System Sciences. 2012.

[17] McMillan, Kenneth L. “Circular Compositional Reasoning about Liveness.” CHARME. 1999.

[18] Namjoshi, Kedar S., and Richard J. Trefler. “On the Completeness of Compositional Reasoning.” CAV. 2000.

[19] Kim, Yubin, et al. “Towards a Science of Scaling Agent Systems.” arXiv:2512.08296. 2025.

[20] De Giacomo, Giuseppe, and Moshe Y. Vardi. “Linear Temporal Logic and Linear Dynamic Logic on Finite Traces.” IJCAI. 2013.

[21] Lemieux, Caroline, Dennis Park, and Ivan Beschastnikh. “General LTL Specification Mining.” ASE. 2015.

[22] Maoz, Shahar, and Rafi Shalom. “Inherent Vacuity for GR(1) Specifications.” ESEC/FSE. 2020.

[23] Maoz, Shahar, and Jan Oliver Ringert. “On Well-Separation of GR(1) Specifications.” FSE. 2016.

[24] Xiao, Yifeng, and Pierluigi Nuzzo. “Symbolic Temporal Supervision of LLM Agents Using Contracts.” arXiv:2609.18128. 2026.

[25] Trivedi, Harsh, et al. “AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents.” ACL. 2024.

[26] Toro Icarte, Rodrigo, et al. “Using Reward Machines for High-Level Task Specification and Decomposition in Reinforcement Learning.” ICML. 2018.

[27] Camacho, Alberto, et al. “LTL and Beyond: Formal Languages for Reward Function Specification in Reinforcement Learning.” IJCAI. 2019.

[28] Xiao, Yifeng, et al. “Contract-Based Consistency and Availability Analysis for Distributed Cyber-Physical Systems.” MEMOCODE. 2026.